Mnemos — Technical Requirements Document (TRD)

Project: Mnemos — A Zero-Dependency Local Semantic Memory Engine **Event:** Zero Dependency | 72-Hour Hackathon (Hackathon Raptors) **Track:** D — Data & Storage
Companion Document: Mnemos_PRD.md (defines *what* and *why*; this document defines *how*)
Last Updated: August 31, 2026

1. Purpose and Scope

This TRD specifies the technical architecture, tech stack, algorithms, data formats, and engineering practices required to build Mnemos within a strict 72-hour, zero-third-party-dependency constraint. Every design decision below is made under three simultaneous constraints, in this priority order:

Rule compliance — nothing here may require a non-standard-library package.
Buildability in 72 hours by a 2-person team.
Maximizing the judged rubric (Functionality 35%, Zero-Dependency Craft 30%, Code Quality 25%, Innovation 10%, plus bonuses).

2. Language and Toolchain Decision

Chosen language: Go.

Criterion	Go	Rust	C
Stdlib coverage for this project (`net/http`, `crypto/*`, `sync`, `bufio`, `encoding/*`)	Excellent, no framework needed	Good but more manual (raw TCP, manual HTTP parsing)	Good but manual memory management adds risk under time pressure
Concurrency ergonomics for WAL/compaction background goroutines	Goroutines + channels, low ceremony	`std::thread` + channels, more boilerplate	pthreads, highest ceremony and highest risk of subtle bugs
Build reproducibility	`go build` with fixed flags is deterministic and simple to verify	Deterministic with `cargo build --locked` (allowed: empty `[dependencies]`)	Deterministic with fixed compiler flags, but more platform-sensitivity
Team risk under a hard deadline	Lowest — garbage collection removes an entire class of bugs (dangling pointers, use-after-free) that would otherwise eat hours in a hand-built storage engine	Higher — ownership/lifetime friction around a WAL/SSTable engine specifically is a known time sink	Highest — manual memory management in a crash-recovery-critical component is the highest-risk combination in this entire list
Single-binary deployability for the demo	Yes, `go build` outputs one static binary	Yes	Yes, but with more platform-specific build flags

Decision rationale: the storage engine (Section 5) is the project's single most important and highest-risk component. Go's garbage collector removes an entire category of memory-safety bugs from exactly the part of the system where a subtle bug (a dangling pointer into a memory-mapped SSTable, a use-after-free during compaction) would be catastrophic to demo live. This is a deliberate, conservative choice: **the goal is a storage engine that provably survives a kill test, not a flex about which language is hardest.** If both engineers are already fluent in Rust, Rust is an acceptable substitute with no change to the architecture below — the algorithms and data formats are language-agnostic.

Toolchain:

Go 1.23+ (stdlib-only; no `go.sum` entries beyond the module's own path)
`go vet`, `go test -race` (both stdlib tooling, not third-party) for correctness and race-condition verification
No build system beyond `go build` — satisfies the "single documented command" submission rule directly

3. Full Standard-Library Tech Stack

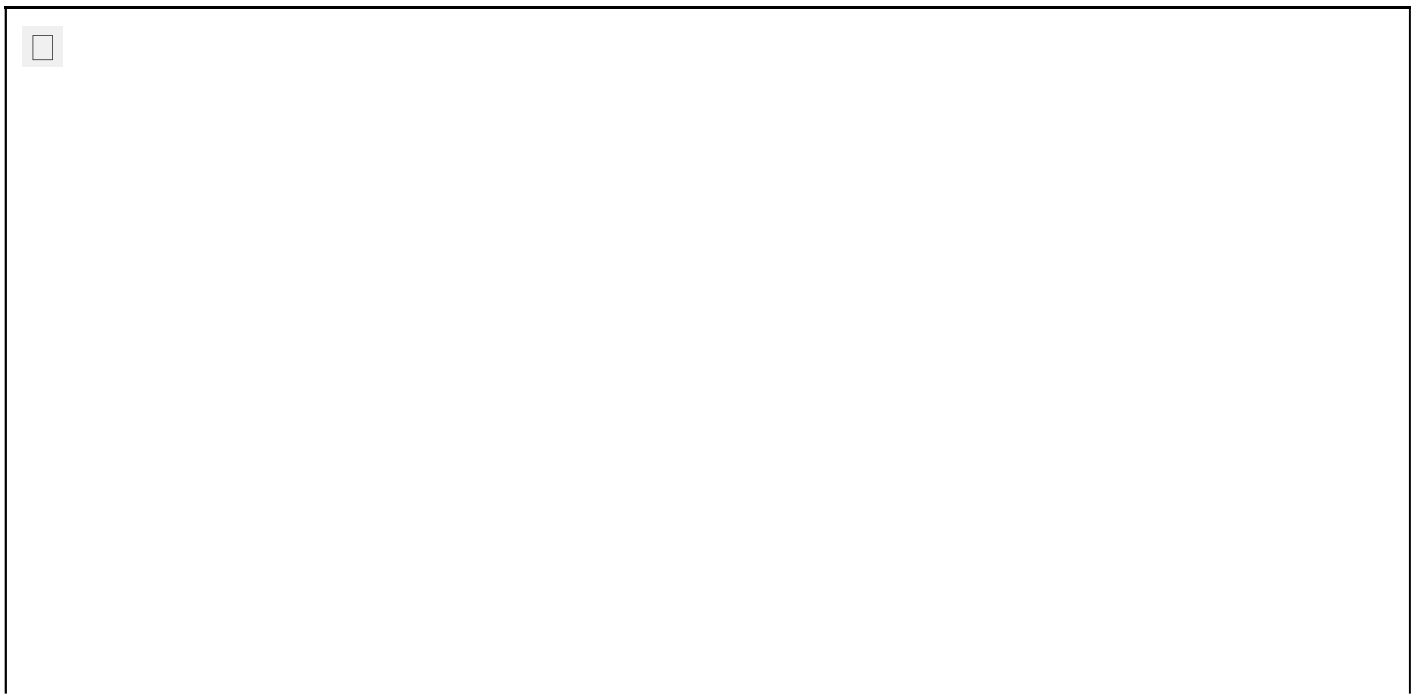
Every package below is part of the Go standard library. No import outside this list is permitted anywhere in the codebase.

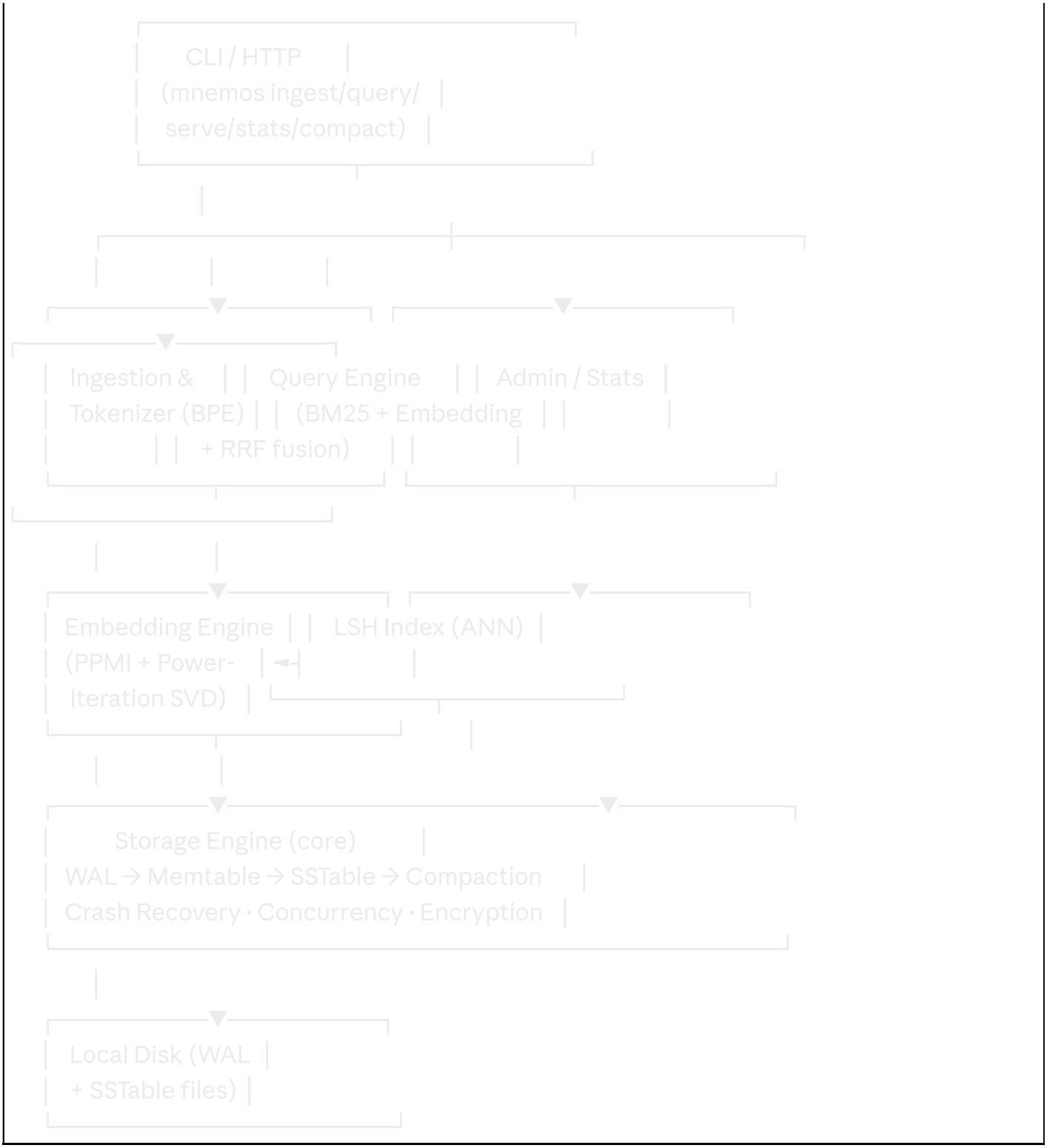
Package	Used for
`net/http`	Local HTTP server and JSON API (F8)
`net`	Low-level socket handling if raw TCP is used for the demo's "impossible mode" stretch goal
`os`	File I/O, process signals, directory walking
`bufio`	Buffered reads/writes for the WAL and SSTable files
`io`	Stream interfaces shared across storage components
`encoding/json`	API request/response encoding, config file format
`encoding/binary`	Fixed-width binary encoding for WAL/SSTable record headers
`sync`	Mutexes, `RWMutex`, `WaitGroup` for concurrency control
`sync/atomic`	Lock-free counters (sequence numbers, stats)
`crypto/aes`	AES block cipher primitive (encryption at rest, F9)
`crypto/cipher`	GCM mode composition around AES
`crypto/sha256`	Hashing (content-addressing, checksums, PBKDF2 building block)
`crypto/hmac`	HMAC primitive, composed into hand-built PBKDF2
`crypto/rand`	Cryptographically secure random bytes (salts, nonces)
`hash/crc32`	Record-level checksums in the WAL and SSTable formats
`unicode/utf8`	UTF-8-aware tokenization boundaries
`unicode`	Character classification for the tokenizer's pre-tokenization step
`strings`, `strconv`	Text utilities
`sort`	Sorting SSTable keys, ranking results
`container/heap`	Priority queue for top-k retrieval during ranking and compaction merge

Package	Used for
`math`	Linear algebra primitives (dot products, norms) underlying PPMI/SVD and TextRank
`math/rand`	Random hyperplane generation for SimHash (seeded for reproducibility)
`flag`	CLI argument parsing
`testing`	Unit, integration, and benchmark tests
`time`	Timestamps, benchmark timing
`context`	Request-scoped cancellation in the HTTP server
`errors`, `fmt`	Error handling and formatting

Explicitly avoided: anything under `golang.org/x/...` (these are *not* part of the Go standard library despite Google's involvement — they are versioned, separately fetched modules and would violate the zero-dependency rule). This includes `golang.org/x/crypto`'s `scrypt`/`argon2` packages, which is why F9's key derivation is a **hand-composed PBKDF2 built from `crypto/hmac` + `crypto/sha256`** rather than importing a KDF package — composition of standard-library primitives, not invention of a new cryptographic algorithm, satisfying Track E's "do not implement your own crypto" norm even outside that track.

4. System Architecture



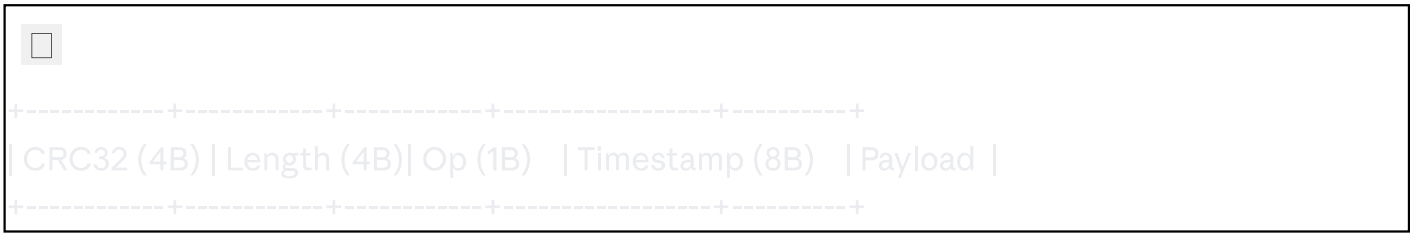


5. Storage Engine — Detailed Design (the core deliverable)

5.1 Write-Ahead Log (WAL)

Every mutation (document insert, index update, delete/tombstone) is serialized and appended to the WAL **before** being applied to the in-memory memtable, guaranteeing durability of any write acknowledged to the caller.

Record format (fixed-width header + variable payload):



`CRC32`: checksum over `Op + Timestamp + Payload`, computed with `hash/crc32`. A corrupt or truncated final record (from a crash mid-write) is detected and discarded during replay — the log is truncated cleanly at the last valid record.
`Op`: `PUT`, `DELETE` (tombstone), or `COMMIT_MARK` (used for grouping multi-key transactional batches).
`Payload`: `encoding/binary`-encoded key/value pair.

WAL writes are `fsync`'d before the write is acknowledged to the caller — this is the actual durability guarantee, not just an in-memory illusion of one.

5.2 Memtable

An in-memory sorted map (implemented as a balanced structure over Go's built-in map plus a maintained sorted key slice, or a simple skip list — no external structure needed) holding recent writes for fast lookups and range scans. Once the memtable exceeds a size threshold, it is frozen (made read-only) and a background goroutine flushes it to an immutable SSTable file while a **new** memtable accepts further writes — this is what allows writes to continue without blocking on disk I/O.

5.3 SSTable (Sorted String Table)

File format:

Data blocks: key-sorted, length-prefixed key/value records, grouped and optionally checksummed per block.
Index block: sparse index mapping keys to data-block file offsets, enabling binary search without loading the whole file.
Bloom filter:* a simple bit-array membership filter (hand-built from `crypto/sha256`/`hash/crc32` as hash functions — not imported) to avoid unnecessary disk reads for keys that definitely don't exist in a given SSTable. *(Stretch goal — cut first under time pressure; the engine is correct without it, just slower.)*

Footer: fixed-size trailer with offsets to the index block and bloom filter, plus a magic number for format validation.

5.4 Compaction

A background process periodically merges multiple SSTables into fewer, larger ones using a k-way merge (`container/heap` as the merge priority queue), dropping obsolete versions and resolved tombstones. This bounds the number of files a read must consult (read amplification) and reclaims space. Compaction runs concurrently with reads via `sync.RWMutex`-protected SSTable manifest swaps: readers always see a consistent view, either the pre- or post-compaction file set, never a torn intermediate state.

5.5 Crash Recovery

On startup:

Load the most recent valid manifest (the list of live SSTable files).
Replay the WAL from the last checkpoint, applying valid records to a fresh memtable, discarding any trailing corrupt/incomplete record (detected via CRC32 mismatch or short read).
Resume normal operation.

Test harness requirement: an automated fuzz test that runs a sustained write workload in a subprocess, sends `SIGKILL` at randomized intervals, and on restart verifies that every write acknowledged before the kill is present and every write in flight at the kill either fully committed or is cleanly absent — never partially applied.

5.6 Concurrency Model

Single-writer, multi-reader per memtable generation (`sync.RWMutex`).
Immutable SSTables require no locking for reads once published, only atomic manifest pointer swaps (`sync/atomic`) to make a newly compacted set visible.
Compaction and flush run on background goroutines coordinated via channels, never blocking foreground query latency beyond a documented bound.

6. Tokenizer — Byte-Pair Encoding

Algorithm (Sennrich et al., 2015):

Initialize the vocabulary as the set of individual bytes/Unicode code points present in the corpus (`unicode/utf8`-aware).
Count all adjacent symbol-pair frequencies across the corpus.
Merge the most frequent pair into a new symbol; add it to the vocabulary.

Repeat for a configured number of merges (default 8,000).
Serialize the ordered merge-rule list and vocabulary to disk.

Tokenization (inference time): apply the learned merge rules greedily, in the order they were learned, to any new input text.

Complexity: training is O(corpus size × merges) with a priority-queue optimization (`container/heap`) to avoid re-scanning the full corpus per merge; inference is near-linear in input length.

Edge cases explicitly handled: empty input, single-character input, mixed-script Unicode input, and unknown-symbol fallback (unseen byte sequences degrade to raw-byte tokens rather than crashing).

7. Embedding Engine — PPMI + Power-Iteration SVD

7.1 Co-occurrence matrix construction

For a sliding context window of size `w` (default 5) over the tokenized corpus, increment a sparse co-occurrence count for every token pair within the window.

7.2 PPMI transform

For each pair `(i, j)`:

```
PMI(i, j) = log( P(i, j) / (P(i) · P(j)) )
PPMI(i, j) = max(PMI(i, j), 0)
```

computed directly from co-occurrence counts using `math.Log`, with add-one smoothing to avoid `log(0)`.

7.3 Power-iteration SVD

Rather than a full SVD (computationally expensive and non-trivial to implement correctly from scratch), Mnemos approximates the **top-k singular vectors** using the power iteration method:

Initialize a random vector `v` (seeded via `math/rand` for reproducibility).
Repeatedly compute `v ← normalize(Mᵀ M v)` until convergence (bounded by a max-iteration count and a convergence-delta threshold).
Deflate the matrix (subtract the found component) and repeat for the next singular vector, up to `k` (default 100) dimensions.

This is standard numerical linear algebra, implemented with nothing but `math` and nested loops over Go slices — no matrix library.

7.4 Document embeddings

A document's embedding is the length-normalized sum of its constituent token embeddings (a standard, simple, defensible pooling strategy).

Complexity: O(vocabulary² × k) for the iterative SVD approximation, bounded by capping vocabulary size via a minimum-frequency cutoff — an explicit, documented performance/quality trade-off.

8. Vector Index — SimHash LSH

Charikar's SimHash (2002): generate `h` random hyperplanes (vectors sampled via `math/rand`, seeded) in the embedding space. For each document embedding, compute an `h`-bit signature by taking the sign of the dot product with each hyperplane. Documents are bucketed by signature (or by signature prefix, for tunable bucket granularity), so that a query only needs to compare against documents sharing its bucket rather than the full corpus.

Complexity: signature computation is O(h × dimensions) per document; query time is O(bucket size), sub-linear in corpus size for a well-tuned `h`.

Tuning: `h` and bucket-prefix length are configurable; the TRD's benchmark suite (Section 12) sweeps these values to report the recall/latency trade-off explicitly in the README rather than hiding it.

9. Ranking — BM25 + Reciprocal Rank Fusion

BM25 is computed directly from term frequencies already gathered by the tokenizer/indexer (standard formula, parameters `k1 = 1.5`, `b = 0.75`, both documented and configurable).

Reciprocal Rank Fusion (RRF): for two ranked lists (BM25-ranked, embedding-cosine-ranked), the fused score for a document is:

Documents are re-sorted by `RRF(d)` descending. This is a published, standard technique for combining heterogeneous rankers and requires no additional dependency — it is pure arithmetic over already-computed rank positions.

10. Extractive Summarization — TextRank

For a retrieved document, split into sentences (rule-based, `unicode`-aware boundary detection), build a sentence-similarity graph (cosine similarity of each sentence's bag-of-embedding-vectors representation), and apply the **same power-iteration code from Section 7.3** to the graph's adjacency structure to compute a PageRank-style centrality score per sentence. The top-ranked sentence(s), up to a character budget, are returned as the result snippet — never generated text.

11. HTTP Interface

`GET /` — minimal HTML query UI (inlined, no external CSS/JS framework or CDN dependency — everything served from the binary itself, satisfying zero-dependency even at the front-end layer).
`POST /api/query` — JSON body `{ "q": "<question>", "k": 10 }`, returns ranked results with per-ranker and fused scores, source document path, and TextRank snippet.
`GET /api/stats` — corpus size, index size, storage engine stats (SSTable count, last compaction time).

Implemented with `net/http`'s `ServeMux` and hand-written handlers — no router library, no middleware framework.

12. Testing Strategy

Test type	Coverage
Unit tests	WAL record encode/decode, SSTable read/write, BPE merge logic, PPMI computation, power-iteration convergence, SimHash bucketing, BM25 scoring, RRF fusion math
Integration tests	Full ingest → query round trip on a fixed reference corpus
Crash-recovery fuzz test	Randomized `SIGKILL` injection during sustained writes (Section 5.5)
Concurrency tests	`go test -race` across all storage-engine and HTTP-handler code paths

Test type	Coverage
Benchmark suite	Query latency at 1k/5k/10k document corpus sizes; LSH recall@10 vs. brute-force baseline; ingestion throughput
Retrieval-quality regression test	Fixed set of query → expected-top-result pairs, run on every build to catch ranking regressions

13. Build & Reproducibility

Single build command: `go build -trimpath -ldflags="-s -w" -o mnemos ./cmd/mnemos` `-trimpath` removes local filesystem paths from the binary, and fixed `-ldflags` strip non-deterministic debug metadata — both are standard, stdlib-toolchain flags, no third-party reproducible-build tooling required.
Reproducible Build bonus verification: build twice from a clean checkout, compute `sha256sum` of both binaries, publish both hashes in the README, confirm they match.
Dependency proof: `go.mod` contents (module declaration only, no `require` block) plus `go build` executed with network access disabled, proving no package fetch is possible or attempted.

14. Security & Threat Model (for F9)

In scope / protected against:

Disk theft or unauthorized filesystem access to the SSTable files at rest (AES-GCM encryption, key derived via hand-composed PBKDF2 over HMAC-SHA256).
Tampering detection via GCM's built-in authentication tag and per-record CRC32 checksums.

Explicitly out of scope (documented, not hidden):

A compromised or memory-inspected running Mnemos process (the decrypted memtable exists in plaintext in memory while running — this is disclosed, not concealed).
Multi-user access control (Mnemos is single-user, single-machine by design; see PRD Non-Goals).
Network-transmitted data (there is no network transmission of document content by design — the HTTP server is local-only).

15. Performance Targets

Metric	Target
Ingestion throughput	⩾ 500 documents/second (plain text, average 1KB each) on reference hardware
Query latency (p95)	< 200ms at 10,000-document corpus
Crash recovery time	< 2 seconds for a WAL under 50MB
LSH recall@10 vs. brute-force	⩾ 90%
Compaction pause impact on read latency	< 10ms added p95 latency during active compaction

16. Directory Structure

```
mnemos/
├── cmd/mnemos/        # CLI entry point (main.go)
├── internal/
│   ├── storage/       # WAL, memtable, SSTable, compaction, recovery
│   ├── tokenizer/     # BPE training + inference
│   ├── embed/         # PPMI + power-iteration SVD
│   ├── index/         # SimHash LSH
│   ├── rank/          # BM25, RRF fusion
│   ├── summarize/     # TextRank
│   ├── crypto/        # AES-GCM + PBKDF2 composition
│   └── server/        # HTTP handlers
├── testdata/          # Reference corpus + fixed query/expected-result set
├── README.md
├── STDLIB.md
└── go.mod             # No require block
```

17. STDLIB.md Substitution Map (build target: ≥ 10 entries)

Normally you'd use	Instead, Mnemos uses
`sentence-transformers` / neural embedding model	Hand-built PPMI + power-iteration SVD (`internal/embed`)
HuggingFace `tokenizers` (BPE)	Hand-built BPE trainer/encoder (`internal/tokenizer`)
FAISS / Annoy	Hand-built SimHash LSH (`internal/index`)
LevelDB / RocksDB / SQLite	Hand-built WAL + SSTable engine (`internal/storage`)
`gensim` (LSA/co-occurrence tooling)	Direct PPMI matrix construction
`scikit-learn` `TruncatedSVD`	Power-iteration SVD approximation
`rank_bm25` (PyPI)	Hand-built BM25 scorer (`internal/rank`)
`sumy` / extractive-summarization libraries	Hand-built TextRank (`internal/summarize`)
`golang.org/x/crypto` `scrypt` / `argon2`	Hand-composed PBKDF2 over `crypto/hmac` + `crypto/sha256`
Web framework (Gin, Echo, Express, Flask)	`net/http` `ServeMux` with hand-written handlers
`gorilla/websocket` or similar, if live-updating UI is added	Plain request/response HTTP polling instead
`viper` / config libraries	`flag` + `encoding/json` for config files

18. Appendix: Algorithm References

Sennrich, R., Haddow, B., Birch, A. (2015). *Neural Machine Translation of Rare Words with Subword Units.* — BPE.

Mihalcea, R., Tarau, P. (2004). *TextRank: Bringing Order into Texts.* — Extractive summarization.

Charikar, M. (2002). *Similarity Estimation Techniques from Rounding Algorithms.* — SimHash/LSH.

Cormack, G. V., Clarke, C. L. A., Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods.* — RRF.

Robertson, S., Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* — BM25.
O'Neil, P., Cheng, E., Gawlick, D., O'Neil, E. (1996). *The Log-Structured Merge-Tree (LSM-Tree).* — Storage engine foundation.